INGENIERÍA DE SISTEMAS · DESARROLLO Y OPERACIONES DE SOFTWARE

ARQUITECTURA Y DISEÑO DEL FRONTEND

Nivel 3: agrupación por funcionalidades y módulos



Imagen de la portada auténtica del prototipo final · commit f72f71d

Escuela Colombiana de Ingeniería Julio Garavito | Sprint 4 | Octubre de 2026

Equipo: Juan Nicolás Álvarez · Camilo Ortiz · Daniel Valero · Juan Diego Valderrama · Paula Alejandra Díaz

VERSIÓN 2.0 Documento actualizado con evidencia de navegación y capturas reales de la versión final. La arquitectura React se presenta exclusivamente como diseño futuro.

00 / Resumen ejecutivo y control del documento

StyleRadar conecta moda local, prendas, tiendas físicas y segunda vida de la ropa en Bogotá. Su demostrador final incluye catálogo, mapa, playlists, perfiles por rol, probador visual, donaciones y gestión administrativa, pero opera con datos simulados dentro del navegador.

LECTURA DE EVIDENCIA N = interacción observada en navegador. C = funcionalidad localizada en el código sin ciclo completo ejecutado. P = arquitectura o integración propuesta. Una pantalla renderizada no implica certificación funcional de sus operaciones.

CAMPO	DEFINICIÓN
Artefacto	Documento de arquitectura y diseño de frontend — versión 2.0
Curso / entrega	DOSW · Sprint 4 · arquitectura nivel 3
Fuente de prototipo	Commit f72f71db7c9e9c6a8f8717e5c3a154c8a9cb932f
Carpeta ejecutable	styleradar-frontend-v3/prototipo/StyleRadar_prototipo/
Estado actual	HTML/CSS/JavaScript + datos locales; no existe aplicación React implementada
Evidencia de QA	95/95 controles de acceso según router, 46 recorridos adicionales, 18 capturas seleccionadas
Manual vigente	docs/StyleRadar_Manual_de_Identidadv2.html · identidad visual del proyecto
Fuente de datos	Inventario técnico aportado por el equipo + revisión documental v1.1

Lo que cambia de la versión 1.1

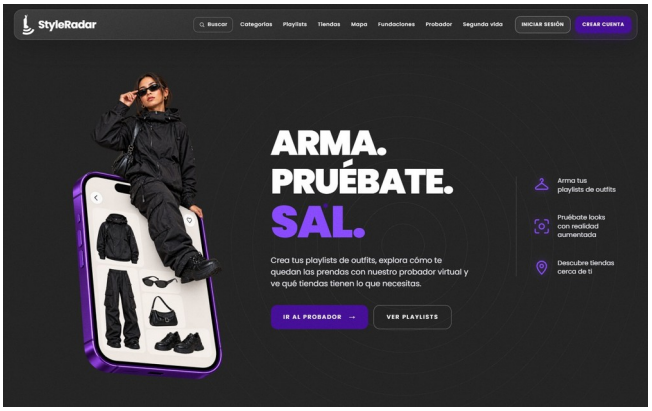
- Se identifica la carpeta final de 198 archivos; se deja de describir el prototipo como un HTML monolítico.
- Se sustituyen las matrices de acceso supuestas por permisos comprobados del router; 95 casos coinciden con el código.
- Se diferencian experiencias simuladas (capas del probador y transacciones locales) de servicios productivos por integrar.
- Se actualizan flujos, capturas y presentación editorial conforme a colores, Poppins, fotografías y personajes de la marca.

01 / Alcance, experiencia y actores

El trabajo conserva los límites del Sprint 4: describir el frontend y su arquitectura modular propuesta, sin introducir una implementación React en el repositorio ni declarar una conexión real con Spring Boot. Las figuras del prototipo muestran comportamientos locales bajo Chrome.



Cinco estilos/personajes del carrusel de portada.



Nueva invitación editorial al probador y a las playlists.

Cinco experiencias de acceso, cuatro roles autenticados

ROL EN EL PROTOTIPO	RESPONSABILIDAD
guest / Invitado	Explorar portada, catálogo, tienda/fundación, mapa, playlist y vendedor públicos.
user / COMPRADOR	Descubrir, guardar, probar, crear playlists, publicar/donar y enviar mensajes.
store / ADMIN_ALMACEN	Gestionar inventario, fotos, colecciones, cuenta, estadísticas, suscripción y mensajes.
found / REPRESENTANTE_FUNDACION	Gestionar perfil institucional, propuestas de donación, mensajes y cuenta.
admin / ADMIN_STYLERADAR	Revisar usuarios, tiendas, fundaciones, publicaciones, playlists y pagos simulados.

MATIZ ESENCIAL El carrusel de cinco personajes (Sastre, Calle, Futuro, Rebelde y Esencial) NO es la taxonomía de seis categorías del catálogo (Formal, Casual, Urbana, Sport, Leather y Calzado).

02 / Arquitectura actual del prototipo

La implementación existente es una aplicación web estática con un documento HTML de entrada y archivos separados de estilos, scripts e imágenes. Los 20 scripts se cargan en orden; comparten objetos globales como SR, SRX y SR_ASSETS, registran vistas por hash y conservan datos simulados en localStorage. No utiliza bundler, módulos ECMAScript o componentes React.

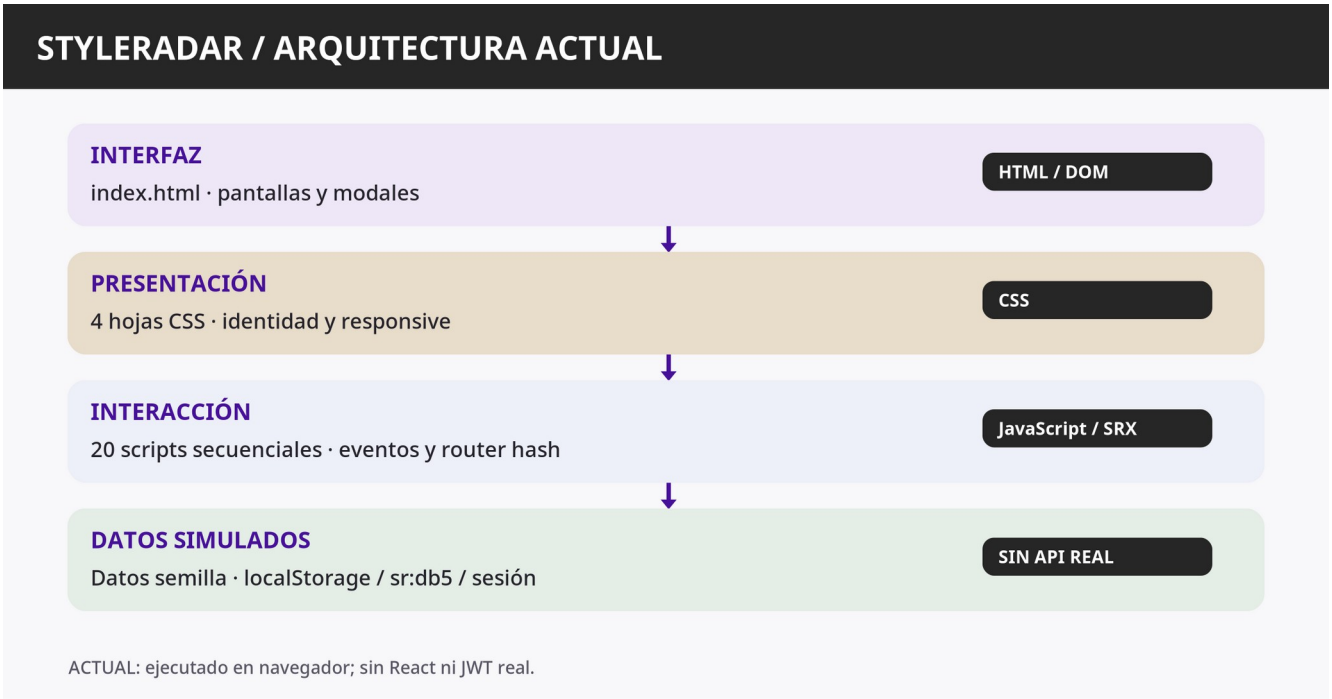


Figura 1. Capas observadas del prototipo real; no representan la arquitectura React futura.

CARPETA / ARCHIVO	RESPONSABILIDAD REAL
index.html	Estructura base, portada y carga ordenada de CSS/JS.
css/ (4 archivos)	Base visual y revisiones sucesivas; version-4.css configura la portada final.
js/05-core.js	Datos semilla, sesión demo, validaciones, router hash, modales y menús.
js/06-user.js	Fichas, playlists, perfil, publicación, mensajes y probador.
js/07-pro.js / 08-admin.js	Gestión de tienda/fundación y administración global.
js/12-req.js	Sustituye catálogo/mapa iniciales: filtros, búsqueda y distancias locales.
assets/img/ (173)	Fotografías, piezas de producto, capas del probador y personajes.

Las extensiones cargadas más tarde pueden sustituir una vista anterior. Ejemplo: catálogo/mapa pasan a la versión de js/12-req.js; el carrusel final de personajes se controla desde js/18-categorias.js. La presencia de código heredado no equivale a una pantalla activa.

03 / Arquitectura propuesta de nivel 3

Se mantiene la decisión académica de agrupar el frontend por funcionalidades/módulos. La futura aplicación React separaría rutas, componentes, estado, modelos/DTO y servicios por dominio; el directorio core contaría con componentes compartidos, diseño y cliente HTTP. Esta es una propuesta de evolución, no una estructura creada dentro del prototipo.

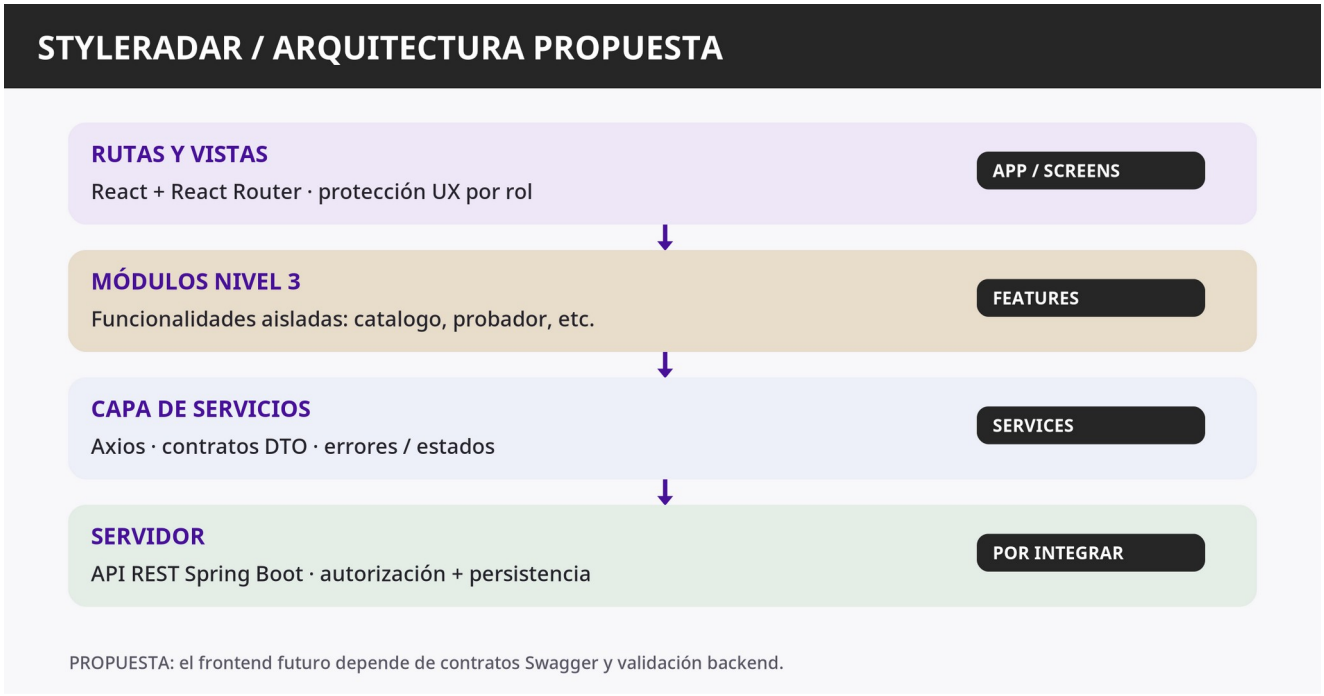


Figura 2. Diseño futuro React/Vite y Spring Boot: por implementar y validar por contratos.

Esqueleto de carpetas propuesto

```
src/
  app/ · rutas y montaje
  assets/ · identidad oficial
  modules/
    core/ · config, design-system, HTTP
    auth/ · discovery/ · catalogo/
    almacenes/ · probador/ · playlists/
    preferencias/ · alertas/ · segunda-vida/
    mensajes/ · fundaciones/
    suscripciones/ · administracion/
```

REGLA DE DISEÑO El frontend no accede directamente a PostgreSQL, JPA o tablas de negocio. La API Spring Boot debe validar permisos y propiedad de recursos en todas las operaciones.

04 / Módulos y contratos internos

Cada módulo propuesto expondría páginas, componentes, hooks, services, models y utilidades solo cuando la complejidad lo requiera. Las dependencias compartidas deben pasar por interfaces explícitas del módulo y el core, sin importar arbitrariamente archivos internos de otro dominio.

MÓDULO	FUNCIONES / RUTAS DEL DEMOSTRADOR	RESPONSABILIDAD FUTURA
core + app	Router hash; navegación; diseño global	Arranque React, rutas, HTTP y componentes comunes
auth	Login/registro mediante overlay	Sesión real, JWT y roles
discovery	Portada; personajes; tarjetas; segunda vida visible	Composición del home y accesos públicos
catalogo	Catálogo; categorías, filtros y modal	Búsquedas, filtros, DTO de prendas
almacenes	Mapa; tienda pública, fotos y localización	Ficha de almacén y búsquedas geográficas
probador	Maniquí y capas de prendas	Visualización y guardado de looks
playlists	Ficha de playlist; guardar/crear/destacar	CRUD y visibilidad de colecciones
preferencias	Perfil, wishlist, planes y cuenta	Datos propios, preferencias y privacidad
alertas	Búsquedas guardadas y avisos locales	Estados y consultas de alertas
segunda-vida	Publicar/donar, vendedor, seguimiento	Publicaciones y propuestas de entrega
mensajes	Bandeja y conversaciones simuladas	Mensajería entre actores
fundaciones	Perfil, panel y recepción de donaciones	Gestión institucional y trazabilidad
suscripciones	Planes, cancelación y pagos locales	Contrato de planes, cobros y estados
administracion	Moderación, actores, playlists y pagos	Administración auditada con API

Estructura interna opcional

components/ para piezas propias; pages/ para pantallas; hooks/ para interacciones; services/ para HTTP y traducción de errores; models/ para tipos y DTO; states/ para estado compartido; utils/ para funciones puras. Los módulos no manipulan la base de datos ni comparten estado implícitamente mediante variables globales.

05 / Matriz real de navegación por rol

La evidencia de navegación comprende 95 intentos: 19 familias de ruta × cinco perfiles. «Sí» representa vista mostrada por el router local; «No» representa redirección o petición de login. No acredita autorización de backend ni protección contra llamadas directas a una API.

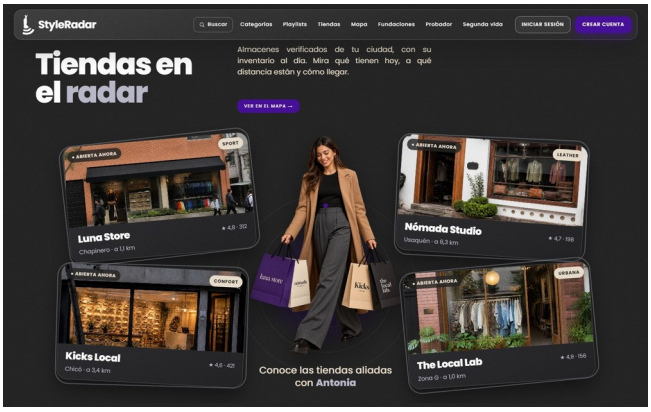
FAMILIA DE VISTAS	INV.	COMP.	TIENDA	FUND.	ADMIN
Inicio / secciones	Sí	Sí	No	No	No
Catálogo / categoría	Sí	Sí	No	No	No
Tienda / fundación pública	Sí	Sí	No	No	No
Mapa	Sí	Sí	No	No	No
Playlist	Sí	Sí	Sí	No	No
Vendedor / cómo funciona	Sí	Sí	No	No	No
Manual de identidad	Sí	Sí	Sí	Sí	Sí
Publicar / donar	No	Sí	No	No	No
Mensajes	No	Sí	Sí	Sí	No
Probador	No	Sí	No	No	No
Mi perfil	No	Sí	No	No	No
Mi tienda	No	No	Sí	No	No
Cuenta profesional	No	No	Sí	Sí	No
Mi fundación / donaciones	No	No	No	Sí	No
Administración	No	No	No	No	Sí

CORRECCIÓN FRENTE A 1.1 La portada, el catálogo y el mapa NO están habilitados para tienda, fundación o administrador. La ficha de playlist SÍ admite al rol tienda. El manual interno #/marca está disponible para todos los tipos de cuenta.

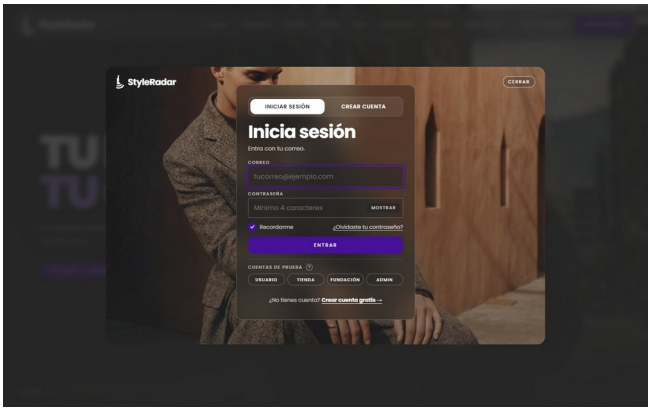
06 / Experiencia de visitante y comprador

6.1. Visitante: descubrir sin sesión

La portada invita a recorrer personajes, playlists, tiendas, fundaciones, mapa y catálogo. Al intentar guardar prendas, publicar, escribir a un vendedor o usar el probador aparece un aviso y se abre el inicio de sesión superpuesto. El formulario de registro también es una superposición; no hay ruta autónoma de login ni registro.



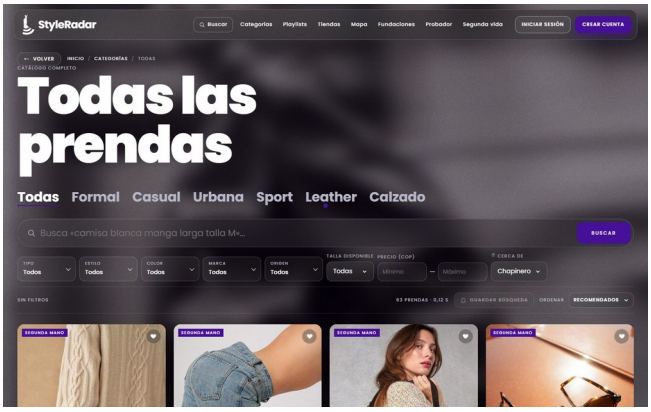
Tiendas para visitante, con Antonia y tarjetas de apoyo.



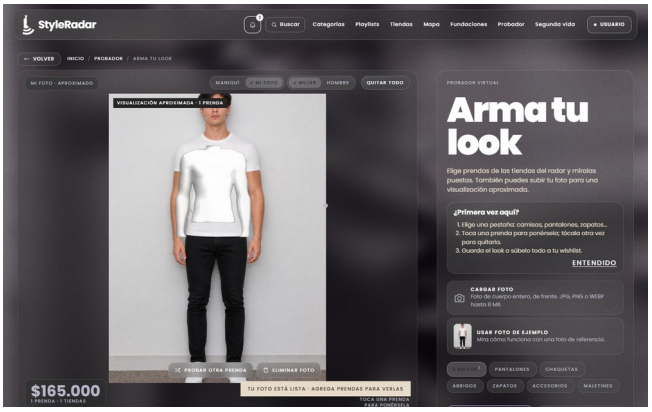
Modal real de acceso al detectar una acción privada.

6.2. Comprador: buscar, guardar, probar y dar segunda vida

Después del login, el comprador puede filtrar prendas, abrir fichas, guardar en wishlist y consultar playlists. El probador superpone capas de prendas sobre un maniquí o fotografía de ejemplo, calcula un total local y permite guardar el look en la demostración. La publicación de prendas y la propuesta de donación se gestionan dentro de #/vender.



Catálogo con filtros y búsqueda local.



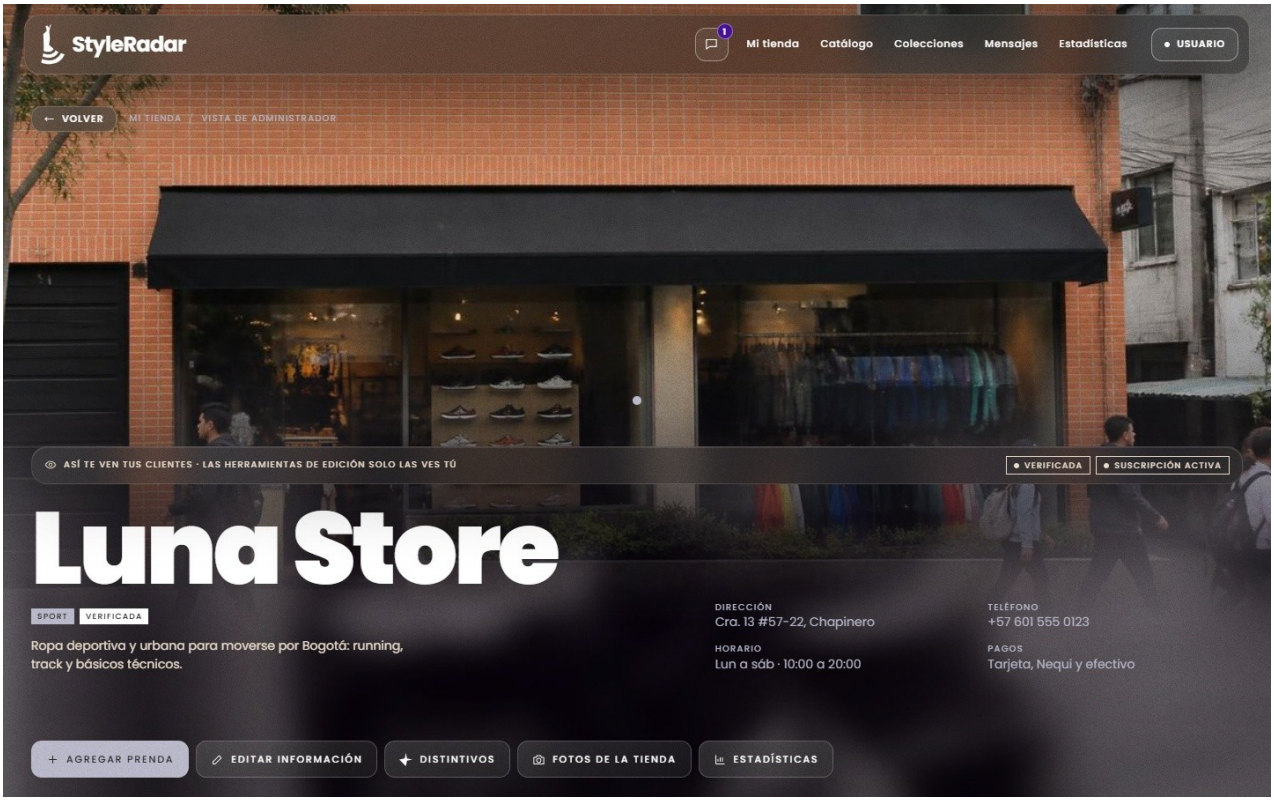
Probador: imagen de ejemplo y capa visual; no IA/RA.

ERROR FUNCIONAL COMPROBADO «Todo a la wishlist» muestra un aviso de éxito, pero la lista permanece idéntica (F10). No debe figurar como operación terminada en el documento ni en Jira.

07 / Experiencia de tienda, fundación y administrador

7.1. Administrador de almacén

La sesión abre #/mi-tienda. El módulo contiene catálogo e inventario, colecciones, fotos del local, creación de prenda mediante asistente de tres pasos, estadísticas, suscripción y mensajes. La ausencia de fotografía bloquea el primer paso del asistente (F12). La carga de hasta seis fotos existe en el código, sin prueba completa de alta.



Vista final de tienda · Luna Store.

7.2. Representante de fundación

La sesión abre #/mi-fundacion; permite revisar solicitudes en #/donaciones, aceptar o rechazar con justificación, coordinar mensajes y consultar información/estado de la cuenta. El rechazo sin motivo se comprobó bloqueado (F13); no se ejecutó la cadena completa de entrega.

07 / Paneles profesionales (continuación)



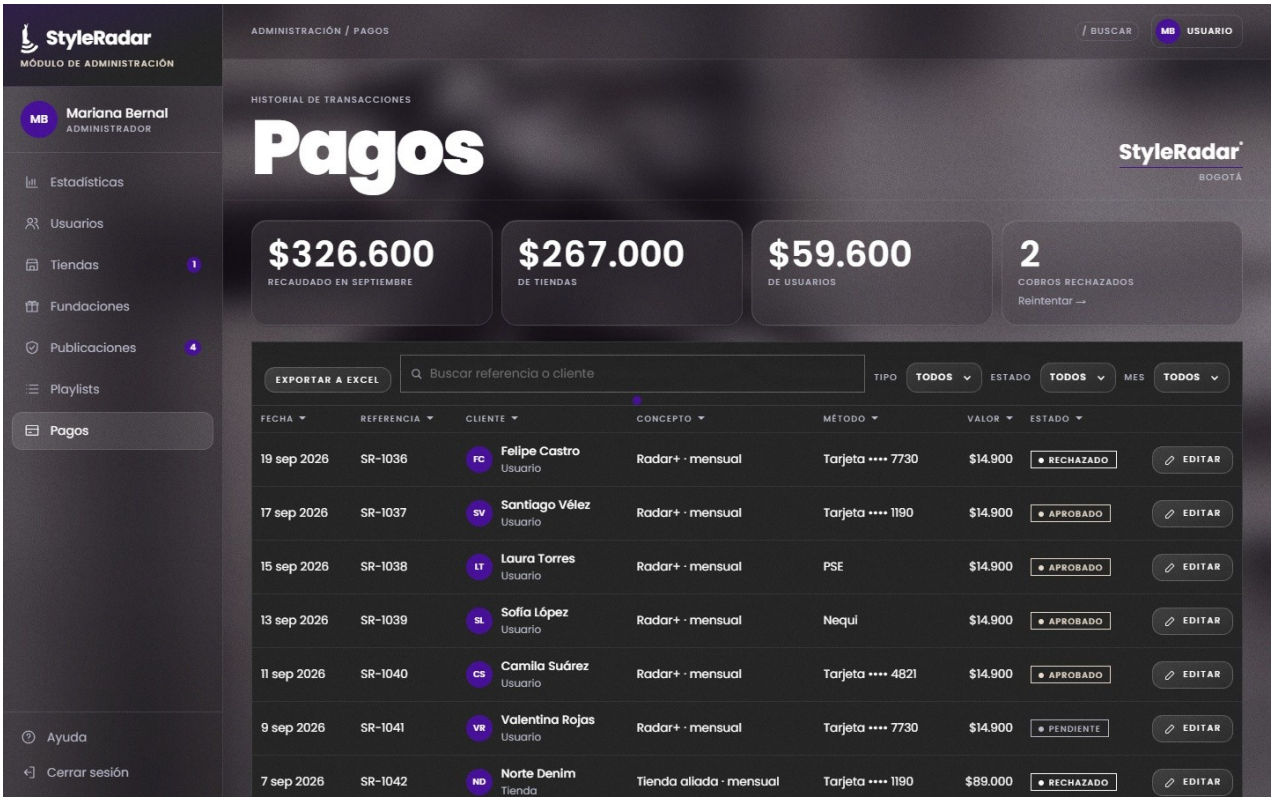
Panel institucional de Fundación Abrigo Bogotá.



Resumen administrativo, cifras locales y gráficas de demostración.

7.3. Administrador StyleRadar

El rol admin entra exclusivamente al panel de plataforma para consultar usuarios, tiendas, fundaciones, moderación, playlists y pagos. Se observó que destacar una playlist modifica el conjunto local de seleccionadas (F14). Los paneles utilizan datos de semilla/localStorage y no estadísticas extraídas del backend.

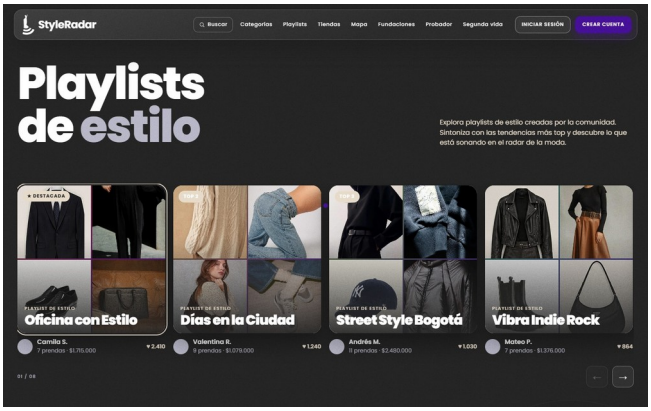


Administración de pagos: tabla local, estados y acciones simuladas.

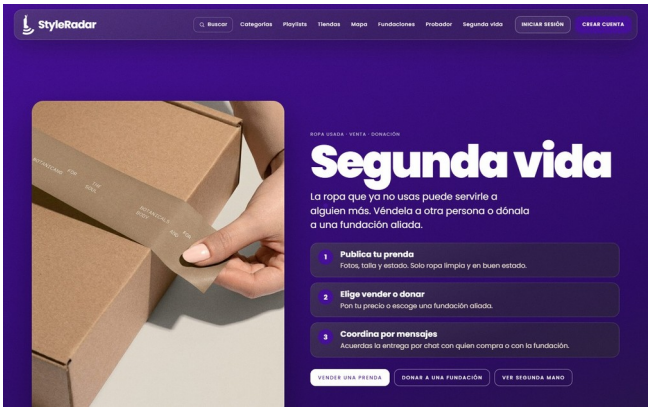
PENDIENTE CONFIRMADO El botón «Exportar a Excel» muestra un toast, pero no descarga ningún archivo XLSX (F15). No se acreditaron reembolsos ni cobros reales.

07 / Galería complementaria de flujos

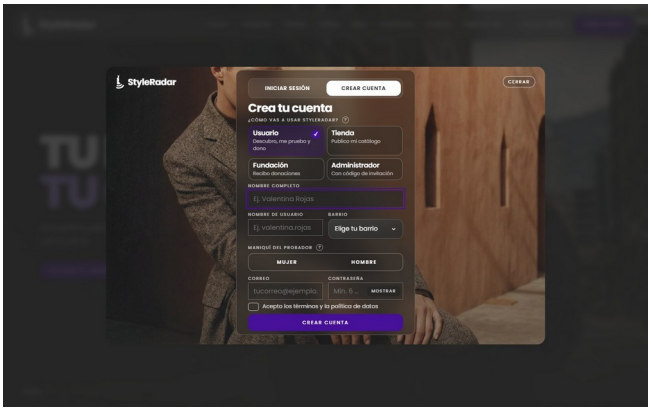
Estas capturas completan la trazabilidad visual de las acciones principales sin sustituir las pruebas: las vistas de registro y perfil son capturas de pantalla, mientras que su persistencia e integraciones completas permanecen pendientes donde lo indica el inventario.



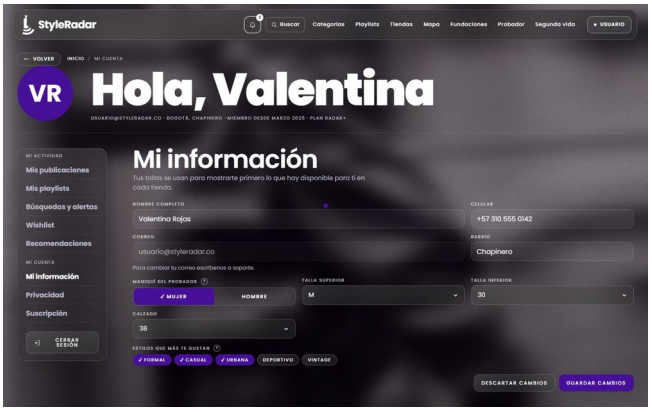
Playlists públicas: carrusel editorial de looks.



Segunda vida: vender, donar y circularidad.



Registro: cuatro tipos de rol, validaciones diferenciadas.



Perfil: navegación a información, wishlist y preferencias.

DECISIÓN DOCUMENTAL Todas las capturas de este documento proceden del navegador y del commit final. No se utilizan mockups históricos para representar la interfaz vigente.

08 / Estado real de las funcionalidades

No debe confundirse «pantalla existe» con «flujo probado». La clasificación siguiente conserva la distinción del inventario técnico, especialmente para requisitos que antes estaban pendientes.

FUNCIONALIDAD	EVIDENCIA DISPONIBLE	ESTADO DOCUMENTAL
Acceso por rol	95/95 rutas coinciden con permisos del código	N: interfaz local validada; backend pendiente
Catálogo y filtros	Búsqueda «camisa» = 2; búsqueda inexistente = vacío	N: casos específicos; filtros combinados C
Wishlist por corazón	Se añade prenda y persiste tras recarga	N: funciona en localStorage
«Todo a la wishlist»	Aviso, pero lista sin cambios	N: incompleta; defecto F10
Probador virtual	Capa visual, total $165.000, look guardado	N: aproximación por capas; no IA/RA
Playlist 15 prendas	MAXPL=15 y opción deshabilitada al llenarse	C: control hallado, límite extremo no probado
Tienda: hasta seis fotos	Validación de máximo seis en código	C: no se completó ciclo de carga
Tienda: wizard	Requiere fotografía en el paso 1	N: error validado; flujo total C
Donaciones	Rechazar requiere motivo	N: validación; ciclo recepción C
Mensajes	Bandeja accesible a user/store/found	N: navegación; envío extremo a extremo C
Playlist destacada	p1 se añade a lista destacada local	N: selección local, no backend
Pagos y estados	Tabla de transacciones y filtros	N: consulta local; cobros/reembolso C
Exportación XLSX	Cero descargas al pulsar	N: incompleta
Móvil	Falta viewport meta; innerWidth=980 en dispositivo 390	N: defecto de escalado
Sesión «Recordarme»	Usuario autenticado clasificado como invitado sin recordar	N: detector de portada inconsistente

09 / Tecnologías, integración y contratos futuros

CAPA	ESTADO Y CRITERIO
HTML / CSS / JS	Implementado en el prototipo final de 198 archivos.
Datos locales / SRX	Simulación: autenticación demo, seed, persistencia en localStorage y reglas de interfaz.
React / Vite	Propuesto; sin proyecto React en la entrega actual.
React Router	Propuesto para sustitución del router hash; rutas a verificar por rol.
Axios / API REST	Propuesto; deben acordarse endpoints y DTO con Swagger/OpenAPI del backend.
JWT + permisos	Propuesto; la autorización efectiva corresponde a Spring Boot, no a ocultar menús.

Diseño de flujo de una operación protegida

La pantalla reúne criterios → el servicio del módulo valida y serializa el DTO → el cliente HTTP añade credenciales a la API configurada, si corresponde → Spring Boot comprueba identidad, rol, titularidad y reglas de negocio → responde con resultado o error; la UI presenta loading, error, empty o éxito sin perder el borrador.

Respuestas y estados esperados en la aplicación React

HTTP	PRESENTACIÓN ESPERADA
400	Mostrar errores de validación junto a campos; mantener valores.
401	En login: corregir credenciales; en sesión: invalidar identidad y solicitar acceso.
403	Explicar falta de permiso sin exponer recursos privados.
404	Informar recurso ausente y regresar a listado.
409	Explicar conflicto/duplicidad; recargar o modificar antes de reenviar.
5xx / red	Mensaje seguro y reintento cuando la operación no duplique efectos.

NO ATRIBUIR AL BACKEND Los errores HTTP, mecanismos JWT, refresh, CORS y endpoints del documento v1.1 son criterios de integración futura; la auditoría actual no ejecutó ni certificó Spring Boot.

10 / Seguridad, privacidad y riesgos

La protección visible en el router del prototipo limita rutas, pero no constituye autorización del lado servidor. El inventario demuestra cuentas demo y contraseñas en claro dentro de sr:db5; es aceptable describirlo como simulación académica, nunca como práctica productiva recomendada.

RIESGO	EVIDENCIA / MEDIDA
Contraseñas en localStorage	F17: demo con claves en claro. El producto final requiere backend y gestión segura de sesión.
Sesión sin «Recordarme»	F16: el detector de la portada confunde usuario activo con invitado. Corregir con la fuente real SRX.role().
Adaptación móvil	M01: ausencia meta viewport; se observó ancho CSS 980 para pantalla 390.
Accesibilidad HTML	Falta doctype, BackCompat y meta charset. Requiere corrección y pruebas posteriores.
Probador con fotografía	Foto de ejemplo y composición local. La foto personal implica consentimiento, retención y borrado futuros.
Cobros, reembolsos y exportes	Simulados. No presentar que existen pasarelas ni descargas Excel reales.
Autorización futura	Comprobar rol y propiedad de recursos por solicitud en API; nunca confiar solo en el cliente.

Reglas de diseño defensivo para React

- No guardar JWT ni credenciales en localStorage, documentos, consultas o logs; preferir access token en memoria y mecanismo de renovación acordado con backend.
- Gestionar loading, vacío, validaciones, errores y expiración de sesión, con mensajes accesibles y persistencia controlada del borrador.
- Diferenciar fotos temporales del probador de fotos publicadas; definir controles de privacidad y autorización antes de subir archivos reales.
- Evitar que una UI muestre confirmación de éxito si el estado no cambió o el servidor no confirmó la operación.

11 / Identidad visual aplicada al documento

El manual vigente define una marca editorial, urbana y centrada en prendas, tiendas y diversidad de estilos. La composición de este Word adopta la jerarquía de títulos, espacios respirables, fotografías originales y elementos gráficos de la paleta, sin sustituir evidencias por ilustraciones genéricas.

Sistema de color 60 / 30 / 10

Grafito #262626	Base, contraste y fondos oscuros
Blanco #FFFFFF	Texto en oscuro y páginas claras
Arena #E8DCCB	Superficies de producto y notas
Lavanda #B9B9CA	Secundario y apoyo
Violeta #48119A	Acento principal, títulos y acciones

Tipografía oficial: Poppins. El DOCX indica Poppins en sus estilos editables; Word utilizará la fuente instalada en el equipo o una sustituta si no está disponible. No se redistribuyen archivos de fuentes. La jerarquía impresa adapta los tamaños del manual a un documento A4 académico.

Los cinco amigos de StyleRadar

PERSONAJE	ESTILO	USO RELEVANTE
Antonia	Sastre	Tiendas y descubrimiento local
Mateo	Calle	Playlists y cultura urbana
Aura	Futuro	Probador y experimentación visual
Nora	Rebelde	Segunda vida y expresión propia
Tomás	Esencial	Fundaciones y propósito



Captura auténtica: carrusel de cinco personajes y estilo de tipografía.

12 / Responsive y accesibilidad: evidencia y correcciones

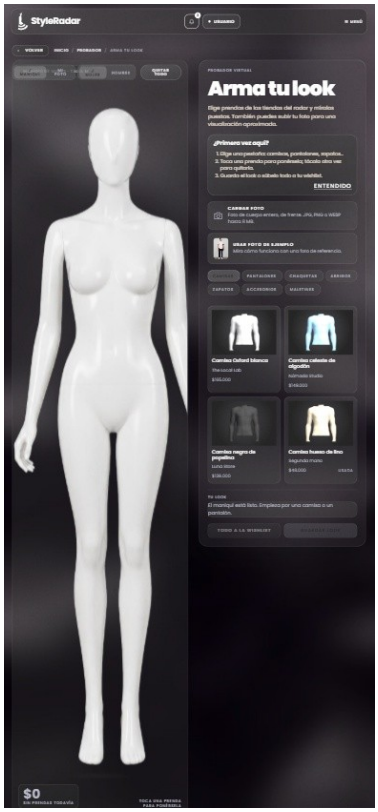
Codex distinguió entre un viewport CSS de 390 px en Chrome desktop y una emulación móvil auténtica con mobile=true. El primero prueba media queries con ancho reducido, pero no subsana que un teléfono reconozca 980 px CSS debido a la ausencia de meta viewport.



Portada · CSS 390



Catálogo · CSS 390



Dispositivo emulado · defecto

Resultado M01: pantalla física 390 px, window.innerWidth = 980 CSS px, meta viewport inexistente. El HTML tampoco declara doctype; document.compatMode = BackCompat. Estos resultados justifican una corrección del prototipo, no solo una afirmación de diseño responsive.

PRIORIDAD ALTA Agregar <!doctype html>, <meta charset="utf-8"> y <meta name="viewport" content="width=device-width, initial-scale=1"> en el HTML final; repetir pruebas en Chrome móvil emulado y, cuando sea posible, dispositivo real. Registrar el cambio en rama de código separada.

Lista de verificación para próxima auditoría

- Contraste, foco, nombre accesible y navegación por teclado; no se realizó auditoría WCAG integral.
- Etiquetas y errores asociados al control, posibilidad de volver a formularios y estados vacíos accionables.
- Media queries reales con <meta viewport>, sin desplazamientos ni texto cortado en menús/modales.

13 / Validación, evidencias y límites de cobertura

ENSAYO	RESULTADO OBSERVADO	INTERPRETACIÓN
Sintaxis JS	20/20 node --check	No cubre todas las rutas de ejecución
Recursos	173/173 imágenes decodificadas	No prueba asignaciones semánticas
Accesos	95/95 según router	Autorización del cliente, no del servidor
Rutas extra	46 visitas	Navegación, no ciclos CRUD completos
Ingresos demo	8/8 correctos	Cuatro roles × dos tamaños
Catálogo	vacío y «camisa» = 2	Casos concretos de búsqueda
Wishlist	corazón persiste	Operación local comprobada
Probador	look y 1 capa	Composición simulada
Donaciones	rechazo sin motivo bloqueado	Validación observada
Destacar	p1 pasa a destacadas	Cambio local demostrado
Exportar	0 descargas	Defecto confirmado
Móvil	innerWidth=980 en pantalla 390	Defecto de escalado

No se ejecutó

No se completaron registro nuevo, publicación completa, carga de fotografía personal, wizard de tres pasos finalizado, ciclo íntegro de donación, envío real de mensajes, reembolsos/cobros, auditoría WCAG, pruebas de carga ni E2E con Spring Boot. La sesión de pruebas empleó preferencia de movimiento reducido para estabilizar las capturas; no valida todas las animaciones.

INTEGRIDAD DE LA EVIDENCIA Según el inventario del equipo, se compararon hashes SHA-256 de 302 archivos originales antes y después; no se editaron prototipo ni recursos durante la revisión. Las 18 JPG son copias seleccionadas de 71 capturas Chrome.

14 / Trazabilidad y próximos pasos

La numeración RF se conserva según el material del proyecto, pero la evidencia disponible no permite declarar cumplimiento integral de 75 requerimientos. Los siguientes son ejemplos de trazabilidad; cada cierre en Jira exige caso ejecutado y criterio de aceptación comprobado.

REQUISITO / CONJUNTO	PROTOTIPO FINAL	ESTADO PARA SPRINT 4
Login y roles	05-core.js; 95 casos de acceso	N: router; API real pendiente
RF29–RF32 · alertas	12-req.js; búsquedas/avisos locales	C: no cierra error 409 backend documentado
Playlists · máximo 15	06-user.js; MAXPL=15	C: límite no ejecutado hasta el extremo
RF72 · destacadas	08-admin.js; F14	N: cambia en local, pendiente integración
RF73 · mensajes	06-user.js; tres bandejas por rol	N: navegación, C: mensaje completo
RF74 · fotos de local	07-pro.js; control de 6	C: carga real completa pendiente
RF75 · pagos	08-admin.js; tabla, estados y F15	N: tabla; exporte no funciona; sin pasarela

Plan de corrección priorizado

- P0 — corregir meta viewport, doctype y charset; repetir emulación de dispositivo.
- P0 — eliminar mensajes de éxito falso: «Todo a la wishlist» y «Exportar a Excel» deben funcionar o marcarse claramente como demostración.
- P1 — corregir detector de tiendas de invitado sin «Recordarme»; contrastar con estado de sesión en memoria.
- P1 — probar límites de playlist, fotos de tienda y ciclos de mensajes, donación y publicación con datos demo aislados.
- P2 — publicar la documentación v2.0 y sincronizar README; el prototipo ya está en main por el PR #2.

- Futuro — acordar contratos del backend y validar autorización servidor, integración JWT, pagos, geodistancias y pruebas E2E.

15 / Fuentes, control de cambios y entrega

Estas fuentes proceden de los materiales de trabajo del equipo, no de una prueba independiente del backend.

- [1] JUANNICOLASA, Frontend-Style-Radar; commit f72f71db7c9e9c6a8f8717e5c3a154c8a9cb932f; ruta styleradar-frontend-v3/prototipo/StyleRadar_prototipo/.
- [2] StyleRadar_Inventario_Tecnico_v2.md, revisión 18/oct/2026, sección de pruebas y anexos A–D (95 permisos y 46 recorridos).
- [3] StyleRadar_Capturas_Seleccionadas_v2.zip, 18 JPG derivados de 71 capturas reales (Chrome 154, escritorio y emulación móvil).
- [4] StyleRadar_Arquitectura_Frontend_Con_Diagramas.docx, versión 1.1, material de base de la actualización.
- [5] styleradar-frontend-v3/docs/StyleRadar_Manual_de_Identidadv2.html, identidad vigente; nombre físico distinto de título interno.
- [6] Escuela Colombiana de Ingeniería Julio Garavito, Sprint #4 — Final · DOSW; presentación semana 11; archivos académicos del proyecto.
- [7] CatrachoINXS, StyleRadar-Backend, y Swagger/OpenAPI referenciados en v1.1; sin nueva comprobación de API para esta edición.

Historial

VERSIÓN	DESCRIPCIÓN	ESTADO
1.1	Arquitectura nivel 3, cinco flujos preliminares y documentación del prototipo anterior.	Antecedente
2.0	Revisión sobre commit final, matriz real de permisos, nueva experiencia visual y hallazgos de QA.	Preparada para revisión del equipo

CONTROL DE PUBLICACIÓN Preparación histórica fuera del repositorio, origen f72f71d. Estado actual: prototipo en main por PR #2, merge 53def7934646c38fcf99d3af65318fa2614e77f2. Este PR solo actualiza documentación; no cierra RF/Jira.

Paquete entregable

1. Word editable y PDF de consulta. 2. Cinco diagramas SVG/PNG por rol. 3. Matriz de cambios v1.1 → v2.0. 4. Propuesta de sustituciones exactas para el README. Todos los materiales utilizan el mismo commit de referencia y declaran las limitaciones de la demo.

A / Anexo · Diagramas de navegación por rol

Los diagramas siguientes expresan el recorrido principal, restricciones del router y resultados de las pruebas. No sustituyen la definición de historias de usuario ni las pruebas E2E del backend. Los SVG/PNG editables están incluidos de manera independiente en docs/flujo/ del repositorio.

A.1 / 01 Invitado

Visitante: entrada libre, catálogo público y login ante acciones protegidas.



Diagrama A.1. Elaboración propia a partir del inventario técnico y la matriz R01.

A.2 / 02 Comprador

Comprador: discovery, playlist, probador aproximado, venta/donación y mensajes.



Diagrama A.2. Elaboración propia a partir del inventario técnico y la matriz R01.

A.3 / 03 Almacen

Tienda: inventario, fotos de local, mensajes y suscripción local.

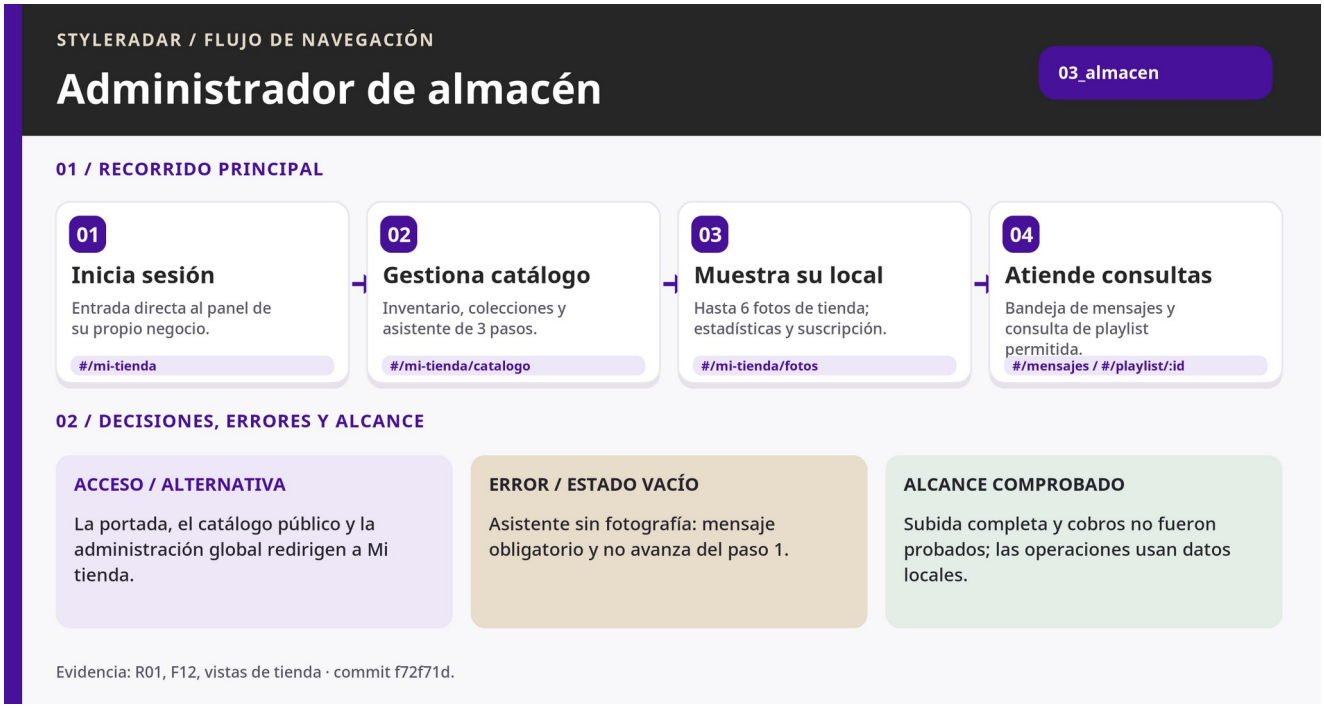


Diagrama A.3. Elaboración propia a partir del inventario técnico y la matriz R01.

A.4 / 04 Fundacion

Fundación: recepción y decisiones de solicitudes con motivo.

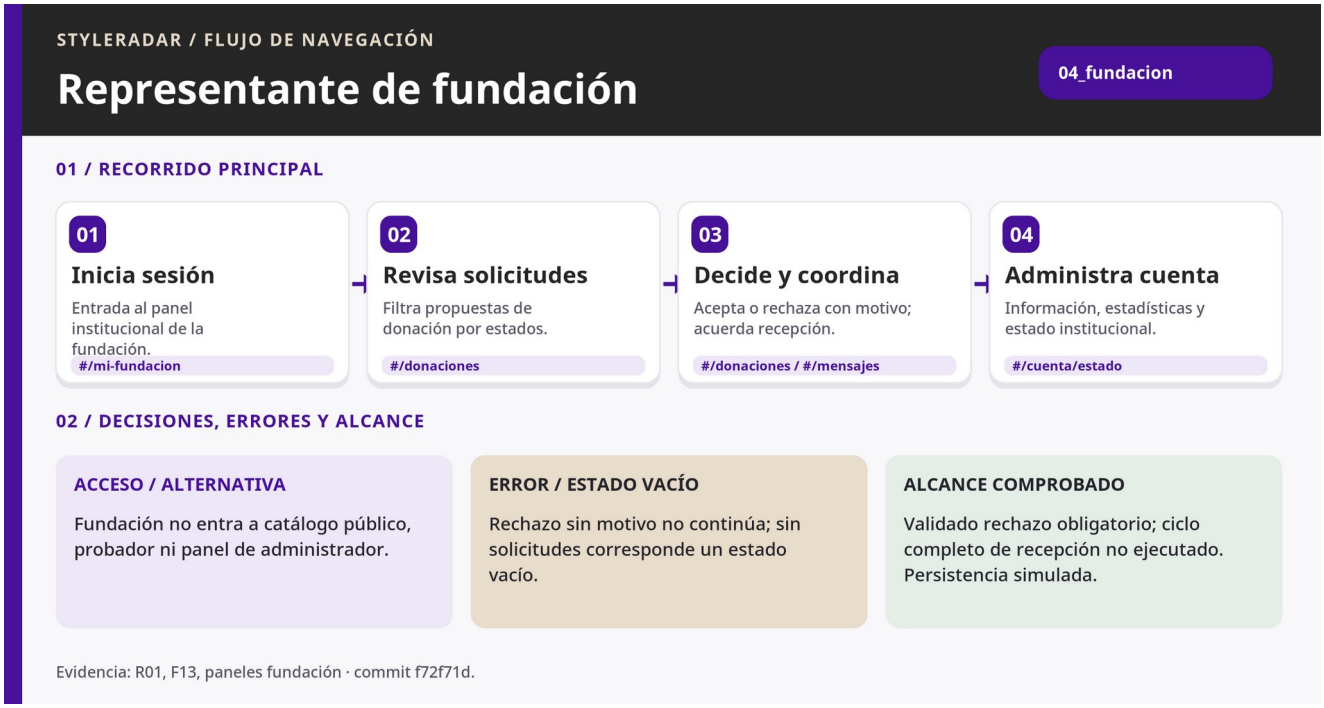


Diagrama A.4. Elaboración propia a partir del inventario técnico y la matriz R01.

A.5 / 05 Administrador

Administrador: actores, moderación, destacadas y transacciones locales.



Diagrama A.5. Elaboración propia a partir del inventario técnico y la matriz R01.